

# Section 06: Loops and Comprehensions **Concept Map**

The full path through this section - we open here before each concept, then dive in.



# Loops: Keywords Covered

Section 06: Loops and Comprehensions - the keywords in bright green are the new ones we cover here.

|                                                           |                                                                              |                                                              |
|-----------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| <div>Values</div> <div>TrueFalseNone</div>                | <div>Classes</div> <div>class</div>                                          | <div>Context Managers</div> <div>with</div>                  |
| <div>Logic &amp; Membership</div> <div>andornotinis</div> | <div>Error Handling</div> <div>tryexceptfinallyraise</div> <div>assert</div> | <div>Cleanup &amp; Flow</div> <div>passdel</div>             |
| <div>Conditionals</div> <div>ifelifelse</div>             | <div>Modules &amp; Imports</div> <div>importfromas</div>                     | <div>Generators &amp; Async</div> <div>yieldasyncawait</div> |
| <div>Loops</div> <div>forwhilebreakcontinue</div>         | <div>Scope</div> <div>globalnonlocal</div>                                   | <div>Soft Keywords</div> <div>matchcase_underscoretype</div> |
| <div>Functions</div> <div>defreturnlambda</div>           |                                                                              |                                                              |

keyword

New in this section

keyword

Covered earlier

keyword

Not covered yet

# Loops: The **while** Loop

A **while** loop runs the same block over and over as long as its condition stays True, so you decide when it stops.

| # | CONCEPT                                                                                                                | SAMPLE CODE                            |
|---|------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| 1 | <b>Count up while a condition holds</b> <code>while</code><br>Loops while the test holds; update the variable to stop. | <code>while count &lt;= 5:</code>      |
| 2 | <b>Loop until false</b><br>Stop when the test fails - no fixed count needed.                                           | <code>value = value * 2</code>         |
| 3 | <b>Count down</b> <code>--</code><br>Subtract each turn to move toward the stop.                                       | <code>n -= 1</code>                    |
| 4 | <b>Running total</b> <code>+=</code><br>Add each value into one running total.                                         | <code>total += i</code>                |
| 5 | <b>Walk a list by index</b> <code>len()</code><br>Step through a list by its index positions.                          | <code>while i &lt; len(scores):</code> |
| 6 | <b>Loop while truthy</b><br>Any truthy value works - a non-empty list is True.                                         | <code>while names:</code>              |

# Loops: What is a **while** Loop?

## What is a while loop?

A **while** loop keeps running the same block **while** its condition is **True**. Something inside must change the condition, **or the loop never stops**.

**WITHOUT a loop** 5 lines for 5 numbers

CODE

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

OUTPUT

```
1
2
3
4
5
```

**WITH a while loop** 4 lines for any count

CODE

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

TURN BY TURN

|           |       |            |           |
|-----------|-------|------------|-----------|
| count = 1 | True  | prints 1   | count = 2 |
| count = 2 | True  | prints 2   | count = 3 |
| count = 3 | True  | prints 3   | count = 4 |
| count = 4 | True  | prints 4   | count = 5 |
| count = 5 | True  | prints 5   | count = 6 |
| count = 6 | False | loop stops |           |

Now print **1 to 100**?

Write **100 print lines**

```
print(1) ... print(100)
```

Change **5** to **100** (1 line code change)

```
while count <= 100:
```

# Loops: The **for** Loop

A **for** loop runs the same block of code once for every item in a list, string, tuple, or other sequence.

| # | CONCEPT                                                                                | SAMPLE CODE                             |
|---|----------------------------------------------------------------------------------------|-----------------------------------------|
| 1 | <b>Loop over a list</b><br>Runs the block once per item, no counter needed.            | <code>for score in scores:</code>       |
| 2 | <b>Loop over a string</b><br>Walks a string one character at a time.                   | <code>for letter in "PYTHON":</code>    |
| 3 | <b>Loop over a tuple</b><br>Reads a tuple's values one by one.                         | <code>for value in (10, 20, 30):</code> |
| 4 | <b>Running total</b> <code>+=</code><br>Add each item into a running total.            | <code>total += score</code>             |
| 5 | <b>Count matches</b> <code>if</code><br>An if inside the loop counts only matches.     | <code>if score &gt;= 80:</code>         |
| 6 | <b>Tuple unpacking</b> <code>for a, b in</code><br>Split each pair into two variables. | <code>for name, score in pairs:</code>  |
| 7 | <b>for vs while</b><br>Use for when you already have the items.                        | <code>while i &lt; len(names):</code>   |

# Loops: What is a **for** Loop?

## What is a for loop?

A **for** loop takes the items **one at a time** and runs the block for each one. You write **no counter** and **no stop test** - it ends by itself when the items run out.

### **WITHOUT a loop** one line per item

#### CODE

```
scores = [70, 95, 60, 88]
print(scores[0])
print(scores[1])
print(scores[2])
print(scores[3])
```

#### OUTPUT

```
70
95
60
88
```

### **WITH a for loop** two lines for any list

#### CODE

```
scores = [70, 95, 60, 88]
for score in scores:
    print(score)
```

#### TURN BY TURN

```
score = 70    prints 70
score = 95    prints 95
score = 60    prints 60
score = 88    prints 88
items run out loop stops
```

## Now the list has **100 scores**?

### Write **100 index lines**

```
print(scores[0]) ... print(scores[99])
```

### Change **nothing** (0 line code change)

```
for score in scores:
```

# Loops: for vs while

Use a for loop when you know the items or the count. Use a while loop when you loop until a condition is met.

## Use for

You have the items or a count - a list, a string, a range.

## Use while

You loop until a condition, and can't list the repeats up front.

## Same list?

If you already have it, for is cleaner - no index to manage.

## FOR VS WHILE

```
colors = ["red", "green", "blue"]  
for color in colors:  
    print(color)
```

```
balance = 1  
while balance <= 100:  
    balance = balance * 2
```

for always ends on its own; while you must stop yourself.

# Loops: The `range()` Function

`range()` builds a sequence of numbers on demand, so a for loop can repeat an exact number of times.

| # | CONCEPT                                                                                               | SAMPLE CODE                          |
|---|-------------------------------------------------------------------------------------------------------|--------------------------------------|
| 1 | <b>Repeat n times</b> <code>range(n)</code><br>Counts 0 up to n-1, so it runs n times.                | <pre>for i in range(5):</pre>        |
| 2 | <b>Start and stop</b> <code>range(start, stop)</code><br>Counts from start and stops before the stop. | <pre>for n in range(2, 6):</pre>     |
| 3 | <b>Step by an amount</b> <code>step</code><br>A step of 2 skips the odd numbers.                      | <pre>for n in range(0, 11, 2):</pre> |
| 4 | <b>Count down</b> <code>-step</code><br>A negative step walks backward.                               | <pre>for i in range(5, 0, -1):</pre> |
| 5 | <b>Sum a run</b> <code>+=</code><br>Add each number into one running total.                           | <pre>total += n</pre>                |
| 6 | <b>See the numbers</b> <code>list()</code><br>range is lazy; list() reveals the numbers.              | <pre>nums = list(range(1, 6))</pre>  |

# Loops: What is `range()`?

## What is `range()`?

`range()` hands the loop a run of numbers, so you never type them out. `range(5)` gives 0, 1, 2, 3, 4 - it starts at 0 and stops before the number you give.

## WITHOUT `range()` type every number

### CODE

```
numbers = [0, 1, 2, 3, 4]
for i in numbers:
    print(i)
```

### OUTPUT

```
0
1
2
3
4
```

## WITH `range()` one number, any count

### CODE

```
for i in range(5):
    print(i)
```

### TURN BY TURN

```
i = 0    prints 0
i = 1    prints 1
i = 2    prints 2
i = 3    prints 3
i = 4    prints 4
5 is NOT included loop stops
```

## Now repeat 100 times?

### Type 100 numbers

```
numbers = [0, 1, 2, ... 99]
```

### Change 5 to 100 (1 line code change)

```
for i in range(100):
```

# Loops: **break** and **continue** (1 of 2)

**break** exits a loop right away and **continue** jumps to the next turn - the keywords that control the flow of a for or while loop.

| # | CONCEPT                                                                                         | SAMPLE CODE                              |
|---|-------------------------------------------------------------------------------------------------|------------------------------------------|
| 1 | <b>Stop the loop early</b> <b>break</b><br>Leave the loop the moment you are done.              | <code>break</code>                       |
| 2 | <b>Break out of while True</b> <b>while True</b><br>The one clean way out of an endless loop.   | <code>while True:</code>                 |
| 3 | <b>Skip to the next turn</b> <b>continue</b><br>Jump to the next item without leaving the loop. | <code>continue</code>                    |
| 4 | <b>Guard with continue</b> <b>continue</b><br>Skip bad values before the real work.             | <code>if score &lt; 80:</code>           |
| 5 | <b>continue in a while loop</b> <b>continue</b><br>Move the counter first or it loops forever.  | <code>n += 1 # move counter first</code> |

# Loops: **break** and **continue** (2 of 2)

**break** exits a loop right away and **continue** jumps to the next turn - the keywords that control the flow of a for or while loop.

| # | CONCEPT                                                                                         | SAMPLE CODE                       |
|---|-------------------------------------------------------------------------------------------------|-----------------------------------|
| 6 | <b>Do-nothing placeholder</b> <code>pass</code><br>Fill a block you will write later.           | <code>pass # fill in later</code> |
| 7 | <b>Loop else runs</b> <code>for...else</code><br>Runs only if the loop never hit break.         | <code>else:</code>                |
| 8 | <b>break skips the else</b> <code>for...else</code><br>A break makes the loop else stay silent. | <code>break</code>                |
| 9 | <b>while else runs</b> <code>while...else</code><br>Runs only if the while ended without break. | <code>else:</code>                |

# Loops: What is **break**?

## What is **break**?

**break** leaves the loop the moment you are done. Without it the loop always walks the whole list, even when the answer turned up on the second turn.

## **WITHOUT break** checks all 4 scores

### CODE

```
scores = [70, 95, 60, 88]
for score in scores:
    print("checking:", score)
    if score >= 90:
        print("found:", score)
```

### OUTPUT

```
checking: 70
checking: 95
found: 95
checking: 60
checking: 88
```

## **WITH break** stops at the answer

### CODE

```
scores = [70, 95, 60, 88]
for score in scores:
    print("checking:", score)
    if score >= 90:
        print("found:", score)
        break
```

### TURN BY TURN

|            |            |              |
|------------|------------|--------------|
| score = 70 | under 90   | keep going   |
| score = 95 | 90 or more | <b>break</b> |
| score = 60 | never runs |              |
| score = 88 | never runs |              |

## Now **1000 scores**, match on turn 2?

### Still checks **all 1000**

998 turns of wasted work

### Stops after **2**

**break**

# Loops: What is **while True**?

## What is a while True loop?

Same job as the last concept - find the first score of 90 or above - but with no **for** loop this time. **while True** never ends on its own, so **break** is the only way out. The older trick is a **flag**, and a flag only stops the loop at the NEXT trip to the top.

## STOP WITH A FLAG the turn still finishes

### CODE

```
scores = [70, 95, 60, 88]
i = 0
running = True
while running:
    if scores[i] >= 90:
        print("first 90+:", scores[i])
        running = False
    print("still checking:", scores[i])
    i += 1
```

### OUTPUT

```
still checking: 70
first 90+: 95
still checking: 95
```

## STOP WITH **break** the turn ends right there

### CODE

```
scores = [70, 95, 60, 88]
i = 0
while True:
    if scores[i] >= 90:
        print("first 90+:", scores[i])
        break
    print("still checking:", scores[i])
    i += 1
```

### TURN BY TURN

```
70  under 90  keep going
95  90 or more break
60  never reached
88  never reached
```

95 is the answer. Did the loop stop when it found it?

**No**

the flag lets the turn finish, so 95 is still checking

**Yes**

break leaves on the line that found it

# Loops: What is **continue**?

## What is **continue**?

**continue** skips the rest of THIS turn and jumps straight to the next item. Take that one line away and Tom falls through to the very line you meant to skip.

**WITHOUT **continue**** Tom is skipped, and still served

### CODE

```
names = ["Sam", "Tom", "Ben"]
for name in names:
    if name == "Tom":
        print("skipping", name)

    print("Give chocolate to:", name)
```

### OUTPUT

```
Give chocolate to: Sam
skipping Tom
Give chocolate to: Tom
Give chocolate to: Ben
```

**WITH **continue**** the turn really does stop

### CODE

```
names = ["Sam", "Tom", "Ben"]
for name in names:
    if name == "Tom":
        print("skipping", name)
        continue
    print("Give chocolate to:", name)
```

### TURN BY TURN

```
Sam  chocolate
Tom  continue skipped
Ben  chocolate
finished all 3 turns
```

One line is the only difference

### without it

Tom gets a chocolate anyway

### with it

the turn ends right there

# Loops: **continue** in a while Loop

## What is the continue trap?

**continue** works in a **while** loop too, but it jumps straight back to the test. If the counter has not moved yet, the loop skips the same value forever.

### COUNTER AFTER **continue** the loop freezes

#### CODE

```
n = 0
while n < 7:
    if n % 2 == 0:
        continue
    print("odd:", n)
    n += 1
```

#### OUTPUT

```
no output
runs forever
n is stuck at 0
```

### COUNTER FIRST skips and moves on

#### CODE

```
n = 0
while n < 7:
    n += 1
    if n % 2 == 0:
        continue
    print("odd:", n)
```

#### TURN BY TURN

```
n = 1    odd: 1
n = 2    even    continue
n = 3    odd: 3
n = 5, 7  odd: 5, odd: 7
n reaches 7    loop ends
```

## Move the counter **after** the continue?

### **n never changes**

frozen on 0, forever

### **n moves first**

so the skip is always safe

# Loops: What is **pass**?

## What is **pass**?

Python needs at least one line inside every block. **pass** is the do-nothing line that fills a branch you have not written yet - it says **empty on purpose**, not forgotten.

**WITHOUT pass** the file will not run

### CODE

```
names = ["Sam", "Tom", "Ben"]
for name in names:
    if name == "Tom":

else:
    print("greet", name)
```

### OUTPUT

```
IndentationError:
expected an indented
block after the if
```

**WITH pass** empty on purpose

### CODE

```
names = ["Sam", "Tom", "Ben"]
for name in names:
    if name == "Tom":
        pass
    else:
        print("greet", name)
```

### TURN BY TURN

```
name = Sam    greet Sam
name = Tom    pass    nothing happens
name = Ben    greet Ben
file runs    the gap is marked
```

## Why not just leave it **blank**?

### **Nothing** runs at all

IndentationError on the empty if

### Runs, and the gap is **deliberate**

**pass**

# Loops: What is **for ... else**?

## What is a **for ... else**?

A **for** loop can carry an **else**. It runs **only when the loop finished without a break** - the "searched everything, found nothing" case. If **break** fires, the **else** is skipped.

## **WITHOUT for ... else** track it with a flag

### CODE

```
names = ["Sam", "Tom", "Ben"]
found = False
for name in names:
    if name == "Amy":
        found = True
        break
if not found:
    print("Amy not found")
```

### OUTPUT

**Amy not found**

## **WITH for ... else** the loop tracks it for you

### CODE

```
names = ["Sam", "Tom", "Ben"]
for name in names:
    if name == "Amy":
        print("found Amy")
        break
else:
    print("Amy not found")
```

### TURN BY TURN

```
name = Sam    not Amy
name = Tom    not Amy
name = Ben    not Amy
no break fired   else RUNS
had Amy been there break, else SKIPPED
```

## Forget the **found = True**?

It reports **Amy not found**  
even when Amy IS in the list

**Nothing to forget**  
else is wired to break itself

# Loops: What is **while ... else**?

## What is a **while ... else**?

The same **else** rule you met on a **for** loop also works on a **while**. Both runs below carry the SAME break - only the number it checks differs. So the else turns on whether the break **FIRE**D, not on whether a break is **there**.

**n == 2, THE BREAK FIRES** else is SKIPPED

### CODE

```
n = 3
while n >= 1:
    if n == 2:
        break
    print("countdown", n)
    n -= 1
else:
    print("finished")
```

### OUTPUT

countdown 3

**n == 4, IT NEVER FIRES** else RUNS

### CODE

```
n = 3
while n >= 1:
    if n == 4:
        break
    print("countdown", n)
    n -= 1
else:
    print("finished")
```

### TURN BY TURN

```
n = 3    countdown 3
n = 2    countdown 2
n = 1    countdown 1
no break fired    else RUNS
prints    finished
```

Same code, same break. So what decides?

The break **FIRED**

n reached 2, so the loop was cut short

The break **never fired**

a countdown from 3 never reaches 4

# Loops: `enumerate()` and Dict Items

`enumerate()` gives each item its position number, and a dictionary can be looped by its keys, values, or key-value pairs with `.items()`.

| # | CONCEPT                                                                                                      | SAMPLE CODE                                       |
|---|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| 1 | <b>Index and item</b> <code>enumerate()</code><br><code>enumerate()</code> numbers each item, starting at 0. | <pre>for index, name in enumerate(names):</pre>   |
| 2 | <b>Start the count at 1</b> <code>start=</code><br>Make a human-friendly numbered list.                      | <pre>enumerate(names, start=1)</pre>              |
| 3 | <b>Loop a dict's keys</b><br>A plain dict loop yields keys in order.                                         | <pre>for field in student:</pre>                  |
| 4 | <b>Key and value</b> <code>.items()</code><br><code>.items()</code> gives the key and value together.        | <pre>for field, value in student.items():</pre>   |
| 5 | <b>Sum the values</b> <code>.values()</code><br>Add every value with a running total.                        | <pre>for score in scores_by_name.values():</pre>  |
| 6 | <b>Pair two lists</b> <code>zip()</code><br><code>zip()</code> lines up two lists side by side.              | <pre>for name, score in zip(names, scores):</pre> |

# Loops: What is `enumerate()`?

## What is `enumerate()`?

`enumerate()` hands the loop the **position and the item together**, so you never keep a counter of your own - and never forget to move it on.

**WITHOUT `enumerate()`** keep your own counter

### CODE

```
names = ["Sam", "Tom", "Ben"]
index = 0
for name in names:
    print(index, name)
    index += 1
```

### OUTPUT

```
0 Sam
1 Tom
2 Ben
```

**WITH `enumerate()`** the position comes free

### CODE

```
names = ["Sam", "Tom", "Ben"]
for index, name in enumerate(names):
    print(index, name)
```

### TURN BY TURN

```
index = 0    name = Sam    prints 0 Sam
index = 1    name = Tom    prints 1 Tom
index = 2    name = Ben    prints 2 Ben
items run out loop stops
```

Forget the `index += 1`?

**Every line shows 0**

```
0 Sam 0 Tom 0 Ben
```

**Cannot happen**

there is no counter to forget

# Loops: Nested Loops

A nested loop is a loop placed inside another loop, so the **inner** loop runs fully on every turn of the **outer** loop.

| # | CONCEPT                                                                                                           | SAMPLE CODE                                      |
|---|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 1 | <b>Loop inside a loop</b><br>The inner loop runs fully on every outer turn.                                       | <pre>for value in row:</pre>                     |
| 2 | <b>Every pair from two lists</b><br>Count of pairs is <code>len(a) * len(b)</code> .                              | <pre>print(letter, number)</pre>                 |
| 3 | <b>Multiplication table</b> <code>range()</code><br>Nested <code>range()</code> builds a numeric grid.            | <pre>print(f"{row} x {col} = {row * col}")</pre> |
| 4 | <b>break in inner loop</b> <code>break</code><br><code>break</code> stops only the innermost loop.                | <pre>if col == 2: break</pre>                    |
| 5 | <b>continue in inner loop</b> <code>continue</code><br><code>continue</code> skips one inner item, keeps looping. | <pre>if col == 2: continue</pre>                 |

# Loops: What is a **Nested Loop**?

## What is a nested loop?

A loop inside a loop, used to walk a grid. The **outer** loop picks the row, the **inner** loop walks that row - so 3 rows of 3 values is  $3 \times 3 = 9$  visits.

### **WITHOUT nesting** one loop per row

#### CODE

```
numbers = [[10,20,30], [40,50,60], [70,80,90]]
for number in numbers[0]:
    print(number)
for number in numbers[1]:
    print(number)
for number in numbers[2]:
    print(number)
```

#### OUTPUT

```
10
20
30
40
50
60
70
80
90
```

### **WITH nesting** same code, any grid

#### CODE

```
numbers = [[10,20,30], [40,50,60], [70,80,90]]
for row in numbers:
    print(row)
    for number in row:
        print(number)
```

#### OUTPUT

```
[10, 20, 30]
10
20
30
[40, 50, 60]
40
50
60
[70, 80, 90]
70
80
90
```

Now the grid has **100 rows**?

**Write 100 loops**

one for every row

**Change nothing**

for row in grid:

# Loops: List, Dict, Set Comprehensions

A **comprehension** builds a list, dict, or set in one line by transforming or filtering an existing collection.

| # | CONCEPT                                                                                                                                   | SAMPLE CODE                                                                                                                           |
|---|-------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>List comprehension</b> <code>[ ]</code><br>Build a list in one line, item by item.                                                     | <code>doubled = [n * 2 for n in scores]</code>                                                                                        |
| 2 | <b>Filter with a test</b> <code>if</code><br>A trailing if keeps only items that pass.                                                    | <code>passing = [n for n in scores if n &gt;= 80]</code>                                                                              |
| 3 | <b>Dict comprehension</b> <code>{k: v}</code><br>Build a lookup of key-value pairs.                                                       | <code>score_map = {n: n * 2 for n in scores}</code>                                                                                   |
| 4 | <b>Filter a dict</b> <code>if</code><br>Add an if to keep only some pairs.                                                                | <code>{n: n * 2 for n in scores if n &gt;= 80}</code>                                                                                 |
| 5 | <b>Set comprehension</b> <code>{ }</code><br>Builds a set and drops duplicate values.                                                     | <code>unique_scores = {n for n in scores}</code>                                                                                      |
| 6 | <b>Pick with if / else</b> <code>if/else</code> <code>list + dict</code><br>Choose between two values per item - in a list, or in a dict. | <code>["pass" if n &gt;= 80 else "fail" for n in scores]</code><br><code>{n: "pass" if n &gt;= 80 else "fail" for n in scores}</code> |

# Loops: What is a **Comprehension**?

## What is a comprehension?

A **comprehension** replaces the empty-list-then-append pattern with a single expression. Read it in the order you say it: **n times 2, for each n in scores.**

THE SHAPE

**[expression for item in collection]**

## **WITHOUT** a comprehension 4 lines and an empty list

CODE

```
scores = [10, 20, 30, 40]
doubled = []
for n in scores:
    doubled.append(n * 2)
print(doubled)
```

OUTPUT

```
[20, 40, 60, 80]
```

## **WITH** a comprehension 1 line

CODE

```
scores = [10, 20, 30, 40]
doubled = [n * 2 for n in scores]
print(doubled)
```

READ IT AS

```
n = 10    10 * 2    20
n = 20    20 * 2    40
n = 30    30 * 2    60
n = 40    40 * 2    80
list is built in one line
```

Same job, **4 lines** or **1**?

### **4 lines**

empty list, loop, append, print

### **1 line**

```
doubled = [n * 2 for n in scores]
```